

Does the answer format impact where CoT will help? *Not much. Free response capabilities required for BigGen Bench may not benefit from pre-planning.*

Many of the commonly-used datasets for problems other than math are multiple choice. We highlight here that CoT has similar performance to direct answer across models for two datasets that are not multiple-choice and contain varying levels of non-symbolic reasoning. First, MuSiQue (Trivedi et al., 2022) is a short-form QA task requiring multi-hop reasoning. We consider this a semi-symbolic dataset as the questions have an explicit multi-hop structure. Because answer spans in MuSiQue can be paraphrased in many different ways, we use GPT-4o to judge if two answer spans are equivalent. Despite being semi-symbolic, we see no overall improvement from CoT.

Second, BiGGen Bench (Kim et al., 2024) uses free-form responses as the answer to a question, and an LLM-as-a-judge is used to evaluate these responses on a scale of 1 to 5. Because free-form responses blur the lines between CoT and direct answering, we create a new prompt that asks the language model to plan the free response before giving it. We then only pass the free response to the judge (GPT-4o-mini in our case) with the prompt from Kim et al. (2024). We also filter out any questions that explicitly state “Think step-by-step”. We plot the performance of BiGGen Bench as the number of times a response receives a score of 4 or better. Despite including many reasoning questions (including several categories of math) and other categories, such as planning, we only see a mild improvement here. Because previous experiments show CoT helping on similar types of questions in the QA format, the lack of similar improvements here could imply that pre-planning is insufficient for unlocking reasoning capabilities in the LLM. Future work is needed to prove this.

Are the gains in Knowledge, Soft Reasoning, and Commonsense significant? *Mostly no, except for MMLU, StrategyQA, and MuSR.*

We tested the significance of the improvements from CoT on the 13 datasets in the Knowledge, Soft Reasoning, and Commonsense reasoning categories using paired bootstrapping to assess whether CoT gives a significant improvement. To account for multiple comparisons, we applied a Bonferroni correction, setting the p-value to 0.00027 to account for the 14 models and 13 datasets. About 32% (59) of the datasets that show a benefit in these three reasoning categories were considered significant. Nearly half of these comparisons (26) are on MMLU and MMLU Pro. On these datasets, we find that CoT is mainly helping on math-related questions. StrategyQA and MuSR also received a consistent performance boost across 10 and 6 models respectively. StrategyQA is often used to benchmark reasoning methods and is built specifically to get a benefit from methods that decompose the question into steps, so a gain in performance is not unprecedented. MuSR, similarly, was built to have multiple steps of complex natural language reasoning, which may receive benefits from CoT. The remaining datasets that receive significant benefits are spread across the datasets and models.

Why do MMLU and MMLU Pro get a boost? MMLU and MMLU Pro contain many different questions requiring different types of reasoning. We separated MMLU and MMLU Pro questions into two bins, those related to math and those not related to math, by checking if the questions text or generated response from the LLM includes an “=”. Figure 4 shows that a majority of the performance gain seen from MMLU and MMLU Pro is from the math slices of each dataset. See more details in Appendix G.

5 STRENGTHS AND WEAKNESSES OF CoT AT FORMAL REASONING

Previous sections establish that CoT primarily helps with symbolic reasoning tasks, but not why. Many symbolic and semi-symbolic tasks be broken down into two stages (Ye et al., 2023; Pan et al., 2023; Jiang et al., 2024): planning, either via a formal or informal specification via prompting (Sun et al., 2024; Wang et al., 2023b), and execution, using the same LM or external solvers. In this section, we attribute the performance gains from CoT on symbolic tasks to these two stages.

Given a question that requires symbolic reasoning, we define the **planning** stage as extracting all variables from the context into a formal specification and defining their relations. The **execution** stage uses a solver that takes as input a plan and can be run in an orderly fashion to derive the final answer. Using our notation from Section 2, let $f(\mathbf{q}) = \mathcal{I}_{\text{planning}}^m(\mathbf{q})$ be a mapping of the question $\mathbf{q}$ to a symbolic plan $\mathcal{S}_{\text{plan}}$ that can be executed by the language model or by an external symbolic solver, $\hat{a} = \text{solve}(\mathcal{S}_{\text{plan}})$, where $\hat{a}$ is the final answer for $\mathbf{q}$.

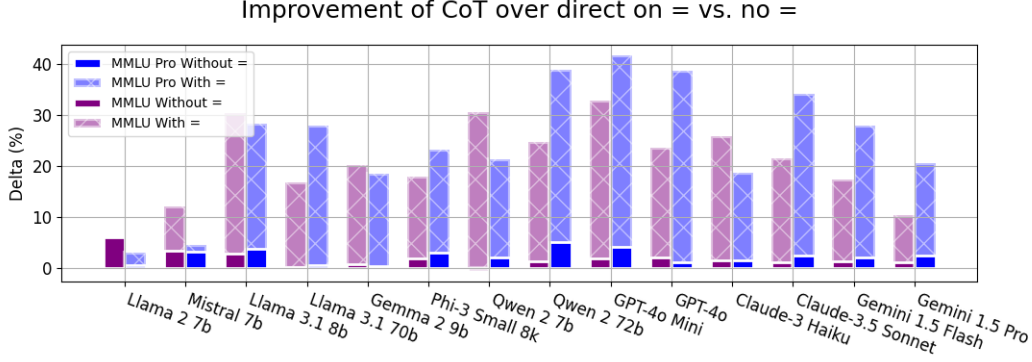


Figure 4: CoT deltas between MMLU and MMLU Pro performance when a question or generated response contains an “=” (With =) or not (Without =). We filter out any questions that do not result in a final answer (degeneration, etc.). CoT primarily helps on the pairs of questions and generations that contain an “=”, which indicates math-related questions.

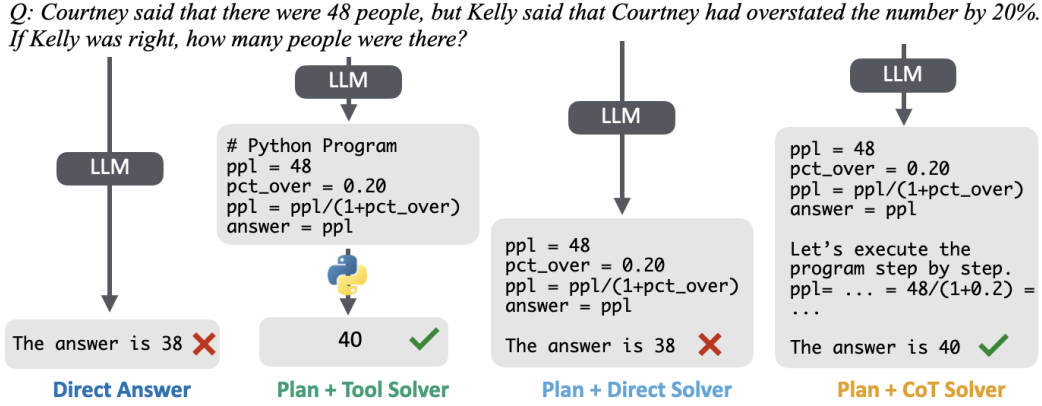


Figure 5: Prompt variants that separate planning and execution for GSM8K. For all prompt variants besides direct answer and CoT (not shown), we few-shot prompt an LLM to first generate a Python program as a solution plan. For Plan + Direct Solver, the LLM is prompted to directly give an answer from the plan; for Plan + CoT Solver, the LLM is prompted to solve the plan step-by-step with CoT and give an answer; for Plan + Tool Solver, we feed the plan into a Python interpreter.

By separating planning and execution in this way, we can test how much a language model can gain from only having a plan, to having a plan and solving it with CoT, or to having a plan and then solving it with an external symbolic solver. Given a plan $\mathcal{S}_{\text{plan}} \sim \mathcal{I}_{\text{planning}}^m(\mathbf{q})$, we compare the performance of the settings below to evaluate at which stage LM is most effective and falls short.

5.1 SETTINGS EVALUATED

Settings 1 and 2: Few-shot direct answer and CoT: We use the few-shot direct answer and CoT prompts from Section 4.1 as baselines. Figure 5 includes an example of each setting on GSM8K.

Settings 3 and 4: Plan + Direct Solver and Plan + CoT Solver: Here we use inspiration from Xu et al. (2024a) and generate a symbolic plan using the same strategy as Ye et al. (2023). Specifically, we use a few-shot prompt $\mathcal{I}_{\text{planning}}^m$ to generate a formal specification $\mathcal{S}_{\text{plan}}$ that should be executable by a symbolic solver. In the same prompt LMs are asked to solve their generated specification $\mathcal{S}_{\text{plan}}$ and derive the final answer $\hat{\mathbf{y}} \sim p(\mathbf{y} \mid \mathcal{I}_{\text{da}}(\mathcal{S}_{\text{plan}}))$, either directly giving the answer after generating the specification (**Plan + Direct Solver**) or providing step-by-step explanations and tracking of intermediate steps for the derivation (**Plan + CoT Solver**). Particularly, $\mathcal{S}_{\text{plan}}$ is a Python program for math datasets, and is a set of first-order logic specifications for logical reasoning datasets.

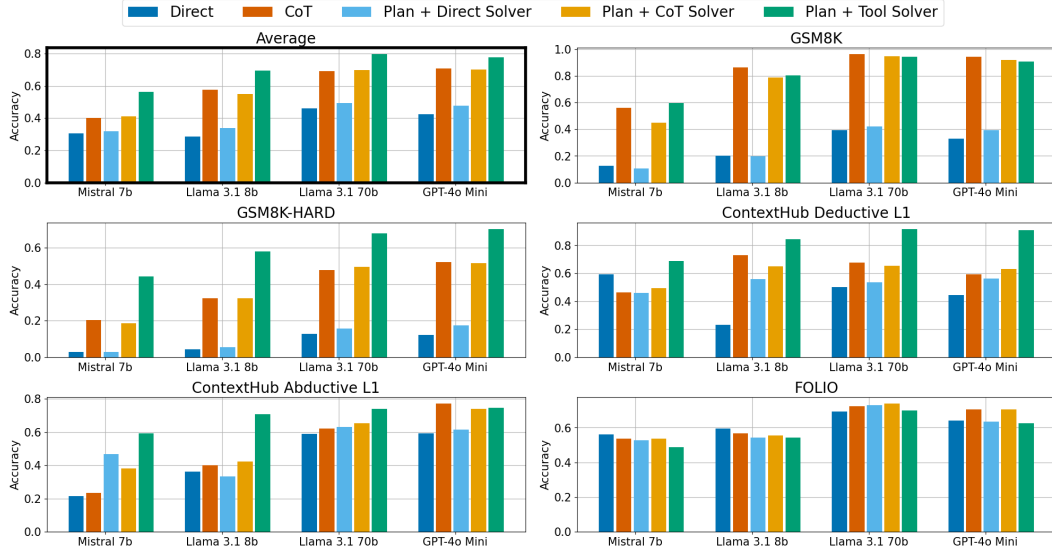


Figure 6: Performance of prompt variants that separate planning and execution for math and logical reasoning datasets. Despite outperforming direct answer for solving a formal plan and deriving the final answer, CoT is still limited in performing symbolic computations: there is a large performance boost from Plan + Tool Solver over CoT and Plan + CoT Solver on average across all models.

Setting 5: Plan + Tool Solver: We then evaluate how effective CoT can be at performing symbolic computations compared with external symbolic solvers. Following prior work on augmenting LMs with tools for math and logic questions (Ye et al., 2023; Pan et al., 2023; Gao et al., 2023; Chen et al., 2023), we generate $\mathcal{S}_{\text{plan}}$ the same way as in CoT Solver, but now feed in the plan into a symbolic solver (Python interpreter or a SMT Solver), such that $\hat{a} = \text{solve}(\mathcal{S}_{\text{plan}})$.

Evaluation Setup: We compare the performance of each setting on math (GSM8K) and logical reasoning (ContextHub and FOLIO) datasets. We follow Gao et al. (2023) to include GSM8K-Hard, a minimally modified version that replaces numbers of GSM8K with larger numbers, to account for the possibility of recent LLMs overfitting GSM8K by data contamination (Zhang et al., 2024).

For Plan + Direct solver and Plan + CoT solver, we use the few-shot prompts from Ye et al. (2023). For Plan + Tool solver, we use state-of-the-art tool-augmented prompting methods. Particularly, for GSM8K, we use Program-aided Language Model (Gao et al., 2023, PAL) that executes the LM-generated plan with a Python interpreter. For logical reasoning datasets, we use Satisfiability-Aided Language Model (Ye et al., 2023, SatLM) that uses automated theorem prover Z3 (De Moura & Bjørner, 2008) to solve the generated specifications. If the generated plan cannot be parsed by the tool, we use random guessing when the question is multiple choice, and mark it incorrect otherwise.

5.2 EVALUATION RESULTS

Figure 6 shows the results across a representative selection of models. Detailed numerical results, including the unparseable rates of model-generated plans, can be found in Appendix H.

When comparing direct answer with Plan + Direct solver and Plan + CoT solver, we note that for many datasets and models, only having a plan does not account for most of the performance gain. **Compared with direct answer, CoT or Plan + CoT solver is needed for strong performance. Tracking the execution with one of these methods gives the strongest accuracy benefit, especially for math-heavy datasets.**

Despite their strength over direct answer and Plan + Direct solver, **CoT and Plan + CoT solver are dominated by Plan + Tool solver in most settings.** LLMs are limited by their ability to execute and track steps compared with symbolic solvers.

We argue that these results provide an explanation of why CoT helps on symbolic tasks. While all tasks could feasibly benefit from a detailed description of how to solve each individual question (e.g., a *plan* in the context of this section), CoT only outperforms direct answer when these steps require a substantial amount of tracing and computation. In these settings, we can see clear performance